Memory Dump ? "The True Persistent Secret"

Flag: TVX{m3m0ry_4rt_n0t_pr0mpt_2026}

Challenge (as given): The SOC team detected suspicious activity on an employee's Windows 10 workstation. An incident responder acquired a full physical memory dump before taking the system offline. Mission: perform in-depth memory analysis to recover the true persistent secret. Adversaries leave noise and obvious decoys for automated tools and AI assistants.

Files: memory.rar (2,634,612,629 bytes, Google Drive file ID 1cuUK3tg9Sxag0HUPaXyGqfxrzVQfHVHH), containing a single memory.raw (6,435,323,844 bytes). SHA-256: rar e12a075d341b9809d9198565cc5ae0234ce619c857ca35bf8daafaa6ab433dd3, raw 1947570ef310c4e02c10b848a464a2317a58550f8f5d09557065618924a8a413.

Intuition

A raw strings scan is exactly what the prompt warns about: the dump is seeded with an obvious flag-shaped string in plain sight. The winning move is to inventory *every* flag-shaped candidate in the 6.4 GB image, then decide each one's fate from the mechanism that holds it ? process command lines, registry persistence, services and tasks are all checkable with Volatility, and only one candidate lives in a place that is neither a command line nor a comment: a process environment block.

Step-by-step analysis

1. Image identification (Volatility 3, windows.info)

- Windows 10, build 19041, x64, 4 processors, VirtualBox guest (VBoxService.exe, VBoxTray.exe).
- System time at acquisition: 2026-11-11 20:03:24 UTC.
- Host DESKTOP-DJHF2A7, user windows 10, logon at 19:47:59 UTC.

2. Process triage (windows.pslist, windows.cmdline)

Almost everything is stock Windows. The interesting family hangs under explorer.exe (PID 5072):

PID	Process	Created (UTC)	Note
6708	powershell.exe	19:48:34	interactive shell, parent of everything below
7208	powershell.exe	19:51:28	hidden, -EncodedCommand (see §3)
8032	cmd.exe	19:51:53	idle interactive cmd
2468	svchost.exe	19:58:26	child of 6708, created and exited in the same second
7388	powershell.exe	19:58:25	hidden, -EncodedCommand, identical to 7208

A svchost.exe whose parent is PowerShell (never services.exe) and which dies instantly is the anomaly to chase.

3. The decoy ? rejected with evidence

Both hidden PowerShell processes carry the same Base64 (UTF-16LE) -EncodedCommand. Decoded, it is a one-liner that sleeps for 86,400 seconds with the string TVX{dont_use_ai_for_solving} sitting in a comment. Whole-image scans (ASCII and UTF-16LE) confirm the population of flag-shaped strings:

- TVX{dont_use_ai_for_solving} ? 12 ASCII + 64 UTF-16 copies: the two sleeper command lines, console buffers, and the setup script text.
- TVX{m3m0ry_4rt_n0t_pr0mpt_2026} ? the only other candidate (see §5).

A flag whose only homes are a Start-Sleep comment and the attacker's own notes is bait, exactly as the prompt advertises. Rejected.

4. Persistence sweep ? all clean

Because the prompt says *persistent*, every durable store was checked and eliminated, which also proves the secret does not survive as a file or registry value:

- HKCU/HKLM Run / RunOnce: only OneDrive (HKCU); HKLM empty.
- Winlogon Shell/Userinit, AppInit_DLLs, BootExecute: defaults.
- HKCU and HKLM Environment keys: only the stock variables.
- windows.svcscan: all AUTO_START services are Microsoft / VirtualBox / Edge ? no rogue service.
- windows.scheduled_tasks: only built-in Windows tasks.
- windows.netscan: nothing but SearchApp and Delivery Optimization / Windows Update traffic ? no C2 channel.
- windows.envars over all live processes: stock variables only.

5. The real artifact ? an environment block in freed RAM

The interactive PowerShell's memory still holds the text of the setup script that staged the challenge (recovered from raw pages at physical offsets around 0x3f6e9000 and 0xc6cf9000). Its second stage decodes a Base64 value and stores it as the environment variable SESSION_AUTH_TOKEN of a spawned svchost.exe -k netsvcs process ? that is PID 2468. Real svchost.exe refuses to run outside the Service Control Manager, so the process exited the second it started (pslist shows identical create/exit times) and Volatility's windows.envars returns nothing for it: its address space was already torn down.

The environment block itself, however, is still resident in freed physical pages. Scanning the raw image for the UTF-16LE string SESSION_AUTH_TOKEN= finds three surviving copies (physical offsets 0x17b7d012, 0x1ee7b38a, 0x1ee7c39e). Around the last two sits a complete, genuine Windows environment block ? ALLUSERSPROFILE, APPDATA, CommonProgramFiles, COMPUTERNAME=DESKTOP-DJHF2A7, ComSpec, Path, SystemRoot=C:\Windows, TEMP, USERNAME=windows 10, ? ? with exactly one non-standard entry:

```
SESSION_AUTH_TOKEN=TVX{m3m0ry_4rt_n0t_pr0mpt_2026}
```

Cross-checks that this, and not the sleeper string, is the secret:

1. It is the value the setup stage Base64-decodes (VFZYe20zbTByeV80cnRfbjB0X3ByMG1wdF8yMDI2fQ==) and assigns to SESSION_AUTH_TOKEN.
2. Its storage mechanism (a named token variable inside a process environment block) matches the mission wording; the decoy's mechanism is a comment in a sleeping process.
3. The whole-image census leaves no third candidate.

Solver

solvers/solve_01_memdump.py (stdlib only) streams memory.raw, finds every UTF-16LE SESSION_AUTH_TOKEN= hit, parses the surrounding environment block, requires the standard system variables (SystemRoot, windir, ComSpec) to be present ? which filters out the script-text copies of the variable name ? and prints the token. solvers/solve_all.py runs it. Fresh-copy test of the packaged zip: 1/1.

```
unzip memdump_writeup.zip && cd memdump_writeup
unrar x memory.rar            # or point the solver at an existing memory.raw
python3 solvers/solve_all.py /path/to/memory.raw
```

Lesson

In memory forensics, *where* a string lives is the evidence. A flag in a command-line comment is a decoy planted for strings-driven tooling; a flag carried as a named variable inside a process environment block ? especially one whose process has already exited, so only raw-page carving still finds it ? is the artifact the challenge is actually about. Always census every candidate first, then let the mechanism decide.